॥ श्रीगणेशाय नमः ॥

Linked Lists

```c
struct node {
    int data;
    struct node *next;
};
```



refers to the same data type (node)
(self-referential)

```c
struct node a = { 10, &b };
struct node b = { 20, %c };
struct node c = { 30, %d };
struct node d = { 40, %e };
struct node e = { 50, NULL };
```

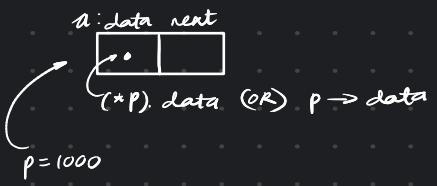



```c
struct node head = { 0, &a };
```

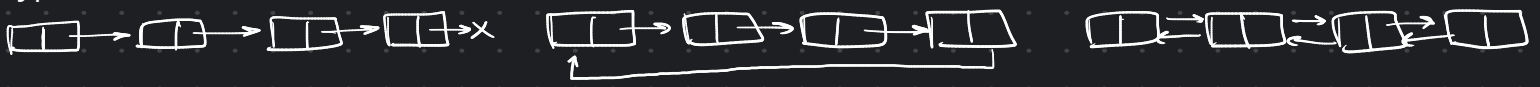



```c
struct node *p = &a;
p->data // refers to 10
p->next // refers to &b
p->next->data // refers to 20
```

OR a.data OR (*p).data



Types of linked lists



Algorithm to find the last node of a singly linked list:

```c
find_last_node(struct node *head) {
    if (head == NULL)
        return NULL;
    struct node *p = head;
    while (p->next != NULL)
        p = p->next;
    return p;
}
```

Algorithm to find the second-last node of a singly linked list:

```c
find_second_last_node(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return NULL;
    struct node *p = head;
    while (p->next->next != NULL)
        p = p->next;
    return p;
}
```

← if only 1 node present in the LL.

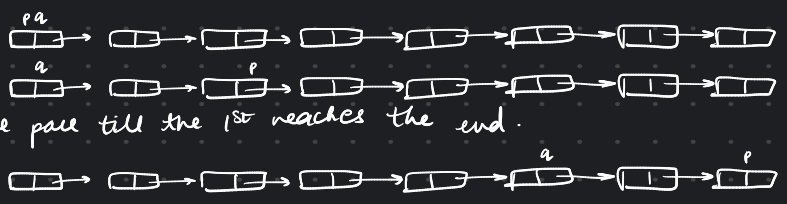
Say we need the element 3rd to the last way.
We could do it in a similar way, but there's a better one.
We could use another pointer that stays a step behind our original pointer. (following pointer)

```c
find_second_last_node(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return NULL;
    struct node *p = head;
    struct node *q = head;
    while (p->next != NULL) {
            q = p;
            p = p->next;
    }
    return q;
}
```

Algorithm to find the kth node from the last of a singly linked list:

1st approach :
- count the # of elements.
- get the position of the kth element from the last $(n-k+1)$ and traverse to that one.

2nd approach :
- keep 2 pointers
- move 1st k places ahead.
- now move both at the same pace till the 1st reaches the end.




```
find_kth_last_node(struct node *head, int k) {
    if (head == NULL) return NULL;
    struct node *p = head;

    while (k != 1 && p->next != NULL) {
        p = p->next;
        k -= 1;
    }

    // if while loop ends and k is not 1, size of list < k
    if (p->next == NULL && k != 1) return NULL;

    struct node *q = head;
    while (p->next != NULL) {
        p = p->next;
        q = q->next;
    }

    return q;
}
```

Algorithm to move the last node of a LL to the beginning

- move the second last node
- set last->next = head->next
- set head = last
- set second-last->next = NULL



```
make_last_node_first(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return head;

    struct node *p = head, *q = head;

    while (p->next != NULL) {
        q = p;
        p = p->next;
    }
    // p becomes last, q becomes second last

    p->next = head;
    q->next = NULL;
    head = p;
    return head;
}
```

← Here we use 2 pointers to do the same thing.

Algorithm to reverse a singly-linked list

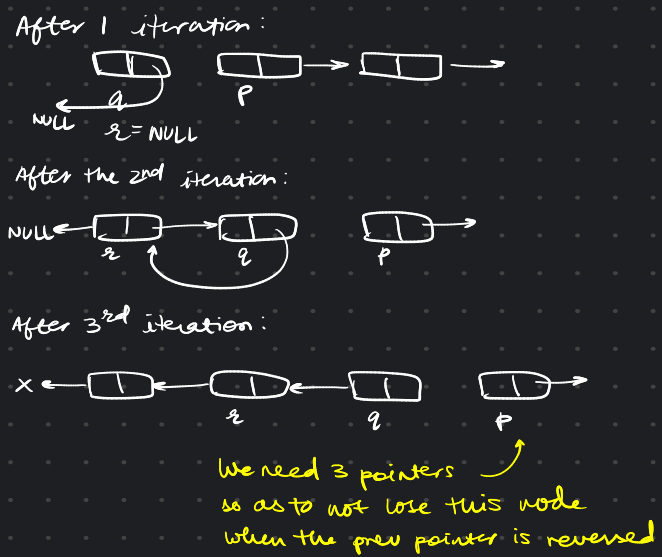
- use 3 pointers.
- flip the pointers of every node to point to the previous node.

```
reverse_linked_list(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return head;

    struct node *p = head;
    struct node *q = NULL;
    struct node *r = NULL;

    while (p != NULL) {
        r = q;
        q = p;
        p = p->next;
        q->next = r;
    }

    head = q;
    return head;
}
```

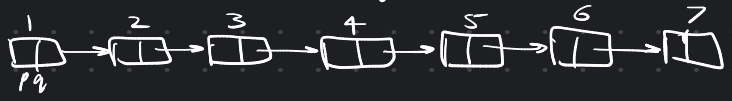


Algorithm to find the middle node in a singly-linked list

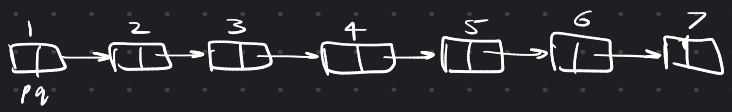
1st approach:
- count the # of nodes
- traverse upto half the count.

] ← 2 passes reqd.

2nd approach:
- assume that the list has an odd # of nodes.



- use 2 pointers: p and q.
- at each step, move p 1 node ahead and q 2 nodes ahead.





when q reaches the end, p will be at the middle since it moves at ½ the rate.

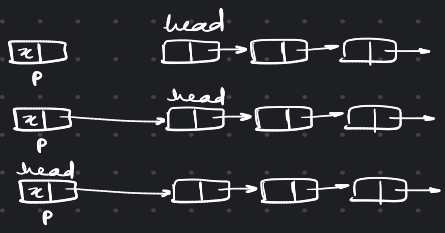
```
find_middle(struct node *head) {
    if (head == NULL) return NULL;
    struct node *p = head, *q = head;

    while (q != NULL && q->next != NULL) {
        q = q->next->next;
        p = p->next;
    }

    return p;
}
```

Algorithm to insert a node at the beginning of a singly-linked list

```c
insert_at_beginning(struct node *head, int x) {
    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->x = data;
    p->next = head;
    head = p;
    return head;
}
```



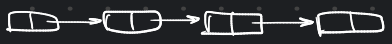
Algorithm to insert a node at the end of a singly-linked list

```c
insert_at_end(struct node *head, int x) {
    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;

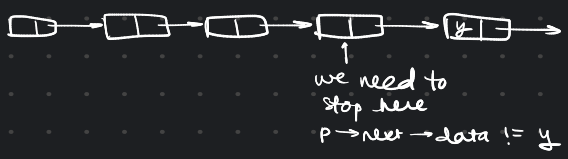
    if (head == NULL) {
        head = p;
        return head;
    }

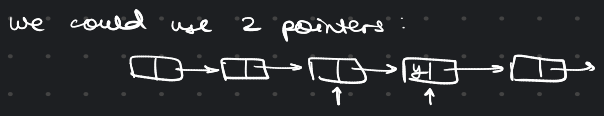
    struct node *q = head;
    while (q->next != NULL)
        q = q->next;

    q->next = p;
    return head;
}
```



Algorithm to insert a node at with data x before a node with data y in a singly-linked list






```c
insert_before_y(struct node *head, int y, int x) {
    if (head == NULL) return NULL;

    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = x;

    struct node *p = head, *q = NULL;

    while (p->next != NULL && p->data != y) {
        q = p;
        p = p->next;
    }

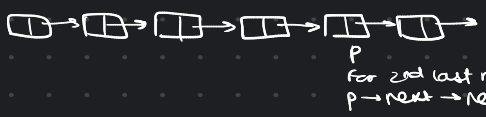
    if (p->data != y) {
        free(new_node);
        return head;
    }

    q->next = new_node;
    new_node->next = p;
    return head;
}
```

if there's no value y in the list.

Algorithm to delete the node at the end of a singly-linked list

- go to 2nd last node
- free (p→next)
- p→next = NULL



for 2nd last node :
p→next→next == NULL

This can be done with 2 pointers.

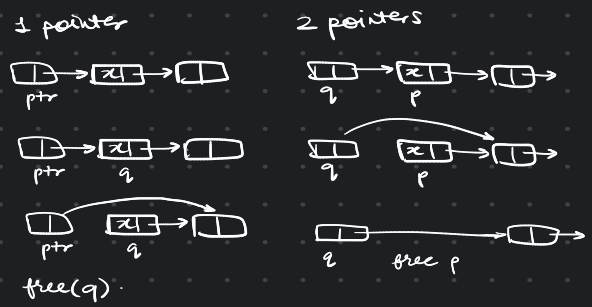
```c
delete_from_end(struct node *head) {
    if (head == NULL) return NULL;
    struct node *p = head, *q = NULL;

    while (p->next != NULL) {
        q = p;
        p = p->next;
    }

    q->next = NULL;
    free(p);
    return head;
}
```

Algorithm to delete the node with data x in a singly-linked list

- use 2 pointers : p to go to x, q to go to a node behind it.
- or use 1 pointer : go to the node just before the node with x.



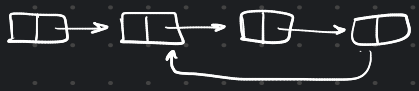

```c
delete_x(struct node *head, int x) {
    if (head == NULL) return NULL;
    struct node *p = head, *q = NULL;

    while (p->next != NULL && p->data != x) {
        q = p;
        p = p->next;
    }

    if (p->data != x) return head;

    q->next = p->next;
    free(p);
    return head;
}
```

Algorithm to detect a cycle/loop in a singly-linked list



Use 2 pointers p and q, one twice as faster than the other.
The faster one, due to the loop, would eventually get behind the slower. But since it is faster, it'll catch up. If there's a cycle, p and q will meet.

```c
bool detect_loop(struct node *head) {
    if (head == NULL) return false;
    struct node *p = head, *q = head;

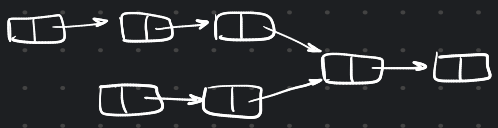
    while (q != NULL && q->next != NULL && q->next->next != NULL) {
        p = p->next;
        q = q->next->next;
        if (p == q) return true;
    }

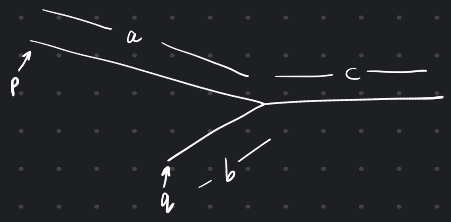
    return false;
}
```

2nd approach :
Use a visited flag for a node.
Keep changing it as we move on.
If for a node, we find that it is previously visited, then there's a loop.
Problem : Extra space : O(n)

```c
struct node {
    int data;
    struct node *next;
    bool visited;  // false by def.
};
```

Algorithm to find the address of the first intersecting node in 2 linked lists





Find a+c, find b+c

$a+c-(b+c)=a-b$

Have p go (a-b) steps.

p and q will then be at equal lengths from the intersection.



Have them move together.

They'll meet at the first intersection, if any.

```c
struct node * find_first_intersection(struct node *head1, struct node *head2) {
    if (head1 == NULL || head2 == NULL) return NULL;
    struct node *p = head1, *q = head2;

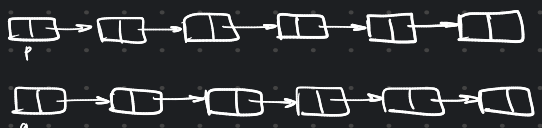
    // find lengths of lists
    int len1 = 0, len2 = 0;
    while (p != NULL) {
        len1 += 1;
        p = p->next;
    }
    while (q != NULL) {
        len2 += 1;
        q = q->next;
    }

    // now find the difference in lengths and have the larger move that diff
    int diff = abs(len1 - len2);
    p = head1; q = head2;
    if (len1 > len2) {
        for (int i = 0; i < diff; i++)
            p = p->next;
    } else {
        for (int i = 0; i < diff; i++)
            q = q->next;
    }

    // now move both together
    while (p != NULL && q != NULL) {
        if (p == q) return p;
        p = p->next;
        q = q->next;
    }

    return NULL;
}
```

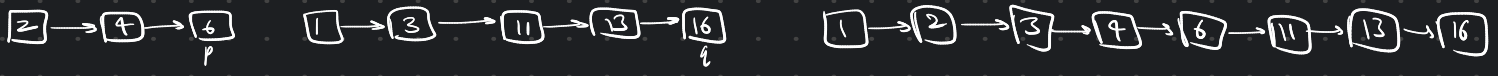
Algorithm to merge 2 sorted linked lists into 1 sorted linked list



Check p and q. Add the smaller to the result. move only the pointer of the smaller one.



Now that one list is exhausted, just travel the rest of the other list



If we need to merge 2 lists L1 and L2 into L1, we need a 3rd pointer for L1.

Drawbacks of singly-linked lists

- we cannot go backwards
- the next pointer of the last node is always NULL

To make use of the last node pointer, we use circular singly-linked lists

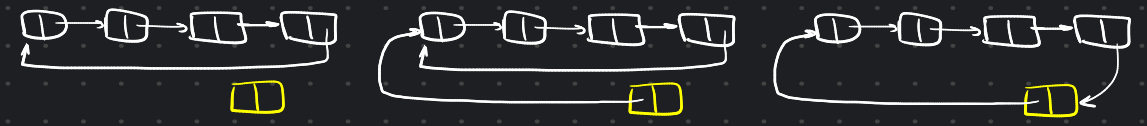
Circular singly-linked lists

The last node points to the first node.



Still, to go a step backwards, we'll need O(n) time.

Algorithm to insert a node at the end of a C-SLL



To find the end, check p->next == head

```c
struct node * insert_at_end(struct node *head, int x) {
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = x;
    new_node->next = head;

    if (head == NULL) {
        head = new_node;
        new_node->next = head;
        return head;
    }

    struct node *p = head;
    while (p->next != head) {
        p = p->next;
    }
    p->next = new_node;
    return head;
}
```

Algorithm to insert a node at the beginning of a C-SLL

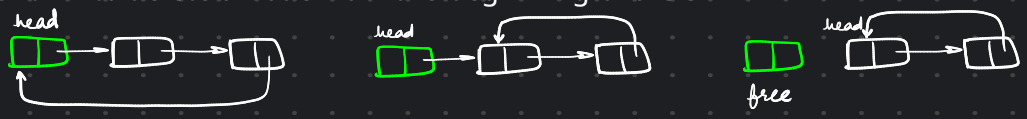
```c
struct node * insert_at_beginning(struct node * head, int x) {
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = x;
    new_node->next = head;

    if (head == NULL) {
        head = new_node;
        new_node->next = head;
        return head;
    }

    struct node *p = head;
    while (p->next != head) {
        p = p->next;
    }
    p->next = new_node;
    return head;
}
```

Going back one step takes O(n) time. Instead, we use doubly-linked lists.

Algorithm to delete a node from the beginning of a C-SLL




```c
struct node * delete_from_beginning(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == head) {
        free(head);
        return NULL;
    }

    struct node *p = head;
    while (p->next != head) {
        p = p->next;
    }
    p->next = head->next;
    free(head);
    return p->next;
}
```

Doubly-linked lists

```
struct node {
    int data;
    struct node *next;
    struct node *prev;
};
```



Going a step back just needs O(1) time.

Algorithm to reverse a DLL

```
struct node * reverse(struct node *head, struct node *tail) {
    struct node *p = tail;

    while (p != head) {
        p = p->prev;
        while (p->next != NULL) {
            printf("%d ", p->data);
            p = p->next;
        }
    }
}
```

Something's amiss.
Check this out.

Algorithm to find the middle node in a DLL

1st approach:
use a fast and a slow pointer.

2nd approach:
count the # of nodes to find the middle.

3rd approach:
use 2 pointers from the either ends.





For even # of elements, check p→next == q.

```
struct node * find_middle(struct node *head, struct node *tail) {
    struct node *p = head, *q = tail;
    while (p != q) {
        p = p->next;
        q = q->prev;
        if (p->next == q) {
            return p;
        }
    }
    return p;
}
```

just for convenience.
If not present,
have q go to the end.

Algorithm to find the address of the second last node in a DLL

```
struct node * find_kth_from_end(struct node *head, int k) {
    struct node *p = head;
    while (p->next != NULL)
        p = p->next;
    for (int i = 0; i < k; i++)
        p = p->prev;
    return p;
}
```

Algorithm to add a node to the end of a DLL




```c
struct node * insert_to_end(struct node *head, int x) {
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = x;
    new_node->next = NULL;

    struct node *p = head;
    while (p->next != NULL)
        p = p->next;

    p->next = new_node;
    new_node->prev = p;
    return head;
}
```

Delete a node with data x from a DLL




```c
struct node * delete_x(struct node *head, int x) {
    struct node *p = head;
    while (p->data != x && p->next != NULL)
        p = p->next;

    // x is not present
    if (p->next == NULL)
        return head;

    // x is first node
    if (p->prev == NULL) {
        head = p->next;
        head->prev = NULL;
        return head;
    }

    // x is last node
    if (p->next == NULL) {
        p->prev->next = NULL;
        return head;
    }

    // x is in between
    p->prev->next = p->next;
    p->next->prev = p->prev;
    return head;
}
```

Circular DLL



To add a node at the end of a C-DLL

```c
struct node * insert_at_beginning(struct node *head, int x) {
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = x;

    new_node->next = head;
    new_node->prev = head->prev;

    head->prev = new_node;
    new_node->prev->next = new_node;

    return head;
}
```

To delete a node from the beginning of a C-DLL

```c
struct node * delete_from_beginning(struct node *head) {
    head->next->prev = head->prev; ①
    head->prev->next = head->next; ②
    struct node *temp = head;
    head = head->next;
    free(temp);
    return head;
}
```

